CHAPTER 5
System Implementation Testing and Evaluation

Intelligent Agent Identity Verification System

5.1 Introduction

This chapter presents the implementation, testing and evaluation of the Intelligent Agent Identity Verification System. The system was developed to register agents, collect identity evidence, combine document and biometric scores, and support verification decisions and controlled access. The implementation comprised a Vue web interface, an Express application server, a Flask artificial intelligence service and a Supabase PostgreSQL database.

The chapter follows a hybrid software and machine learning structure. It explains the implemented modules and data flow, describes the synthetic dataset and model development procedure, and evaluates the recorded functional tests and classification results. The latest saved model, version 3.0.0, achieved 75.800% accuracy on 1,500 fresh synthetic cases. This result measured performance under the simulator assumptions and did not establish operational identity verification accuracy.

5.2 Implementation Environment

5.2.1 Hardware and deployment environment

The implementation was organised as a local web prototype connected to managed Supabase services. The documented development ports were 5173 for the interface, 3000 for Express and 5000 for Flask. Network access was required for Supabase and initial model downloads. The recorded experiment identified Python 3.13, but did not establish a minimum tested processor, memory, storage capacity or client-device specification. Consequently, hardware minimums and public deployment availability were not inferred from software configuration. The project README documented the local setup, while the machine learning README recorded the experiment environment.

Table 5.1 Implementation environment and evidence

Layer	Recorded configuration	Evidence and scope
Client	Browser with camera access	Capture components; no minimum device benchmark recorded.
Application services	Vue, Express and Flask; local development ports	README setup; no measured concurrent-user capacity.
Data services	Supabase authentication, PostgreSQL and storage	Schema and integration code; region and service tier not established.
AI execution	CPU-capable face and liveness inference	Model-loading code; no minimum hardware certification.
Experiment	Python 3.13; saved model v3.0.0	ML README and artifact manifest; isolated offline experiment.

5.2.2 Software services and configuration

The interface used Vue and Vue Router to separate authentication, verification and administrator screens. Express coordinated authenticated requests and database operations. Python provided the image-processing and classification libraries. The package manifests recorded dependency ranges, whereas the experiment lock file and model manifest recorded exact versions for the saved model. These two forms of evidence were distinguished because a declared range does not prove which application package version was tested. These dependencies were documented in the frontend and backend package manifests, the AI service requirements file and the saved model manifest.

Table 5.2 Software dependencies and their roles

Component	Version evidence	Role
Vue / Vue Router	^3.5.13 / ^4.5.0	Interface and navigation
Vite	^6.0.7	Development server and frontend build
Express	^4.21.2	Application API and request handling
Supabase JavaScript client	^2.49.1	Authentication and data integration
SimpleWebAuthn	Server ^14.0.3; browser ^14.0.0	Optional passkey authentication
Flask	3.1.3 in model manifest	AI service endpoints
scikit-learn	1.9.0 in model manifest	SVM, scaling and calibration
NumPy / SciPy	2.5.3 / 1.18.1 in manifest	Numerical processing
OpenCV	>=4.8,<5 in requirements	Image preprocessing and quality metrics
EasyOCR / InsightFace	Unpinned in service requirements	Text extraction and face embeddings
PyTorch / Silent-Face	PyTorch unpinned; local model weights	Presentation-attack scoring

Backend configuration supplied the Supabase URL and keys, AI service URL, listening port and authentication-related settings. Service-role credentials remained server-side. Optional passkeys required the configured relying-party identifier and allowed origins; the project README specified Node.js 22 or newer for this feature. The README also retained an older general Node.js 18 prerequisite, so Node.js 22 or newer was the relevant documented requirement for the complete passkey-enabled application.

The full image-processing environment was distinct from the smaller locked machine learning experiment environment. The latter supported reproduction of the classifier experiment without downloading every OCR, face and anti-spoofing model. Unpinned production dependencies limited exact reproduction of the complete image pipeline. No container-based deployment was established by the evidence reviewed.

5.2.3 Deployment and reproducibility

A reproducible installation required the application source, authorised Supabase access, database scripts, dependency files and the complete saved model bundle. The following procedure describes the repository setup; it does not constitute evidence that a fresh deployment was executed during preparation of this chapter.

1. Obtain the project repository and record the source revision. Create a new Supabase project or use an authorised isolated test project. Apply database/schema.sql only to a fresh database; apply the relevant access-control, audit, date-of-birth, storage, row-level-security and optional passkey scripts according to their prerequisites.

2. Configure the backend and AI service environment files without publishing credentials. Supply the Supabase connection settings, AI service URL and frontend origin. Configure passkey relying-party settings when that feature is enabled.

3. Install backend and frontend packages from their respective package directories. Create a Python virtual environment and install ai-service/requirements.txt for the full AI service. Preserve compatible scikit-learn versions for the supplied model files.

4. Make the InsightFace models and Silent-Face weights available. Retain the complete ai-service/ml/models bundle and the supporting ml.decision_model module. Stop the serving process before replacing a model bundle, then restart it so startup validation can run.

5. Start the backend with npm run dev, the AI service with python app.py, and the frontend with npm run dev. Verify the backend and AI /health endpoints, then inspect /api/svm/status to confirm that the expected model version is loaded.

6. Run isolated tests before demonstrating a lawful sample registration, document capture, selfie capture and verification workflow. Check the resulting request, scores, decision basis and access record in the authorised test environment.

For model reproduction, ai-service/ml/requirements-experiment.lock.txt supplied the exact experiment dependencies. The v3 workflow was invoked through ai-service/ml/improve_model.py --jobs 3 --promote in the documented virtual environment. Reproduction belongs in an isolated copy because promotion changes the active model bundle. The runner refused to repeat a completed experiment against the same evaluation draw. Running the older train_model.py workflow could replace the bundle with v2 artifacts and was therefore not interchangeable with reproducing v3.

Health checks established service responsiveness only. They did not prove that authentication, database permissions, image inference and access enforcement all worked together. The recorded model promotion changed local artifacts; it did not demonstrate publication to a remote hosting service.

5.3 System Implementation

The implemented architecture separated presentation, application coordination, image analysis and persistence. This separation allowed the browser to guide capture while the backend managed protected operations and the AI service produced numerical evidence. The descriptions below trace implemented capabilities rather than assigning unverified Chapter 4 requirement numbers.

5.3.1 Database and data implementation

The relational schema represented users, agent profiles and verification attempts as separate entities. UUID primary keys identified records, while foreign keys linked an agent to a user and each request to an agent and document. Unique constraints on user email, agent user reference and national identity number limited duplicate account and profile records. Verification scores and decisions used unique request references so that each request had one associated score row and decision row. The database schema and accompanying SQL migration scripts defined these entities and constraints.

Table 5.3 Core database entities

Entity	Important contents	Relationship or purpose
users	user_id, email, role	Authentication-linked account and role
agents	agent_id, user_id, national_id	Profile linked uniquely to a user
documents	document_id, agent_id, file_url	Evidence references for an agent
verification_requests	request_id, document_id, status	Tracks a submitted verification
verification_scores	request_id and six scores	One score record per request
kyc_decisions	Decision, probabilities, basis, time	One outcome per request
access_control_records	agent_id, decision_id, access_granted	Records access enforcement
audit_logs	Action, outcome, actor, time, details	Traceable application events

Status values distinguished pending, verified, review and rejected requests. The decision enumeration excluded pending because it represented completed classification outcomes. The decision_basis field distinguished SVM classifications from placeholder threshold decisions. This distinction supported interpretation of stored outputs when model inference was unavailable.

Row-level security was enabled in the base schema, with policies supplied separately. Enabling row-level security alone did not demonstrate that every deployed policy was correct. The backend also validated bearer tokens through Supabase and checked roles before protected actions. Database configuration and server-side checks therefore formed separate parts of the access-control implementation.

The retention implementation removed stored identity images and selfies after three business days following the latest successful verification, while preserving decision and audit metadata. Business days were Monday to Friday in Africa/Nairobi, without public-holiday exclusions. Cleanup ran at startup and at fifteen-minute intervals, and failed storage removals were retried. This policy depended on the backend remaining active; it did not purge pending, review or rejected cases. The document-retention service and the README described this policy.

5.3.2 Application modules and verification workflow

Authentication and profile management supported agent registration, sign-in and completion of identity details. Authentication middleware rejected missing or invalid bearer tokens, and role middleware restricted administrator operations. Optional passkeys supplemented password and Google sign-in. The passkey implementation used WebAuthn credentials rather than storing a user's fingerprint or other device biometric. The authentication and role middleware, together with the project README, documented these controls.

Document and selfie capture components guided evidence collection. The image-quality service calculated sharpness, brightness and contrast, while document preprocessing supported extraction. OCR compared extracted identity information with registered details. Face matching used InsightFace embeddings to compare a document portrait and selfie, and liveness scoring used Silent-Face anti-spoofing models. These produced evidence signals; neither text agreement nor facial similarity independently proved that a document was authentic.

The backend sent evidence to the quality, OCR, face and liveness endpoints and assembled six numerical features. The classifier returned Verified, Manual Review or Rejected, together with class probabilities and model provenance. Backend integration preserved the lowercase database values verified, review and rejected. Decision and access-control records connected the classification to the agent's permitted workflow. Administrator screens supported inspection and human decision-making.

Table 5.4 Classifier feature contract

Feature	Unit or range	Interpretation
face_match_score	0–100	Face similarity score
liveness_score	0–1	Anti-spoofing score
ocr_confidence_score	0–1	Agreement with registered identity text
blur_score	Nonnegative raw value	Laplacian variance; higher means sharper
brightness_score	0–255	Mean HSV value-channel intensity
contrast_score	0–127.5	Grayscale standard deviation

Raw image-quality measurements were retained in their original units. MinMax scaling occurred within the model processing path; generic 0–1 quality ratings were not substituted for the expected raw metrics. Missing, non-finite, wrongly typed or out-of-range evidence was rejected rather than silently converted to zero.

5.3.3 Exception handling and decision provenance

At the AI endpoint, an unavailable or inconsistent model bundle produced an unavailable-service response. At application level, valid complete evidence could instead enter the existing threshold fallback when SVM inference failed. The fallback recorded placeholder_threshold and an audit reason; its zero-or-one outputs were rule outcomes rather than calibrated confidence. Therefore, model-only evaluation results did not measure the combined application, quality gates, fallback and human-review workflow.

5.3.4 Dataset description and preparation

The decision-model dataset contained 10,000 synthetic cases: 6,000 Verified, 2,000 Manual Review and 2,000 Rejected. Each training record contained the six numerical features and a class label. These class quotas were experimental assumptions rather than estimates of operational prevalence. No genuine identity records or invented participant data were required to describe the classifier experiment. The machine learning README, generation configuration and dataset metadata recorded the composition and assumptions of the dataset.

Local benchmark measurements informed parts of the generator. The recorded sources included 1,000 LFW face pairs, 1,200 NUAA presentations, 500 MIDV quality frames and MIDV OCR measurements. Stored measurement hashes and summary statistics were retained. The experiment read these measurements; it did not establish that the original source images had been reprocessed or that historical measurements had been independently verified. These datasets did not supply the final three-class KYC labels.

Face and liveness values were sampled from truncated distributions informed by recorded measurements. OCR and quality generation combined measured statistics with explicit simulation assumptions. Quality measurements used a multivariate distribution over transformed blur, brightness and contrast. Some unresolved and genuine scenarios deliberately shared observed distributions, representing information absent from the six features. Thus, labels represented assumed latent scenarios rather than a simple threshold applied to the observed scores.

Exact and near-duplicate checks preceded partitioning. Row identifiers and latent scenario descriptions were excluded from model predictors. Scaling and optional quality transformations were fitted within training folds, reducing leakage from validation and evaluation records. The duplicate checks screened close observations but did not prove that the simulated population represented real applicants. Dataset ownership, licence conditions and permission to redistribute original benchmark material required separate verification; the chapter makes no redistribution claim.

Table 5.5 Data partitions used in the v3 experiment

Partition	Cases	Function
Training	7,000	Model fitting and five-fold cross-validation
Calibration	750	Probability calibration only
Selection	750	Candidate and decision-policy selection
Original test	1,500	Retired from v3 model selection and fitting
Fresh evaluation	1,500	Frozen candidate and incumbent comparison

Calibration and selection each contained 450 Verified, 150 Manual Review and 150 Rejected cases. The fresh evaluation contained 900, 300 and 300 respectively. A new draw was generated after candidate selection was frozen, and duplicate checks covered all 11,500 original and fresh records. The original test set was retired because previous results had informed the new research questions.

5.3.5 Model training selection and packaging

An RBF support vector machine was used to combine the six features into a three-class decision. Training compared C values of 0.1, 1, 10 and 100, with gamma values of 0.001, 0.01, 0.1, 1, scale and auto. Five-fold shuffled stratified cross-validation used macro F1, balanced class weights and seed 42. This produced 24 parameter combinations per grid. Preprocessing remained inside the cross-validation pipeline. The training scripts and frozen experiment protocol recorded the parameter grid and cross-validation procedure.

The v3 experiment compared raw and transformed quality inputs, native multiclass and explicit one-versus-rest SVM variants, and sigmoid, isotonic and temperature calibration. Three additional grids were fitted, while the prior raw-native winner was reused with provenance. Calibration used only the calibration partition. Candidate selection used the separate selection partition and imposed safeguards on macro F1, review recall and both false-verification counts relative to the comparable calibration baseline.

The selected configuration was raw_native_sigmoid, with C = 10 and gamma = scale. Its recorded training cross-validation macro F1 was 0.672766. Its probability policy used a review multiplier of 1.0, verification threshold of 0.0 and verification margin of 0.0. These settings did not provide an additional conservative verification threshold beyond the selected probability policy.

The release gate required strictly higher fresh-set accuracy, non-decreasing macro F1 and no increase in either Rejected-to-Verified or Manual-Review-to-Verified errors relative to the saved v2 incumbent on the same fresh records. The selected candidate passed this recorded gate and was promoted to the local version 3.0.0 bundle. Passing the gate established the specified comparative condition; it did not establish an acceptable operational false-approval rate.

The model manifest recorded the feature order, class names, library versions and artifact hashes. The service validated the model bundle at startup and mapped class probabilities using the model's class labels rather than assuming a fixed column order. Saved model, scaler and supporting code were required together for reproduction. The model remained static during service operation; completed verification attempts did not trigger online learning. The artifact manifest and SVM service implementation documented these loading and prediction checks.

5.4 Testing and Evaluation

5.4.1 Test strategy and acceptance criteria

The recorded evaluation combined isolated classifier-contract tests, experiment-integrity checks, backend decision tests and a held-out synthetic model comparison. Functional assertions checked valid outputs, invalid inputs, model unavailability and preservation of decision provenance. The experiment assessed accuracy, macro and weighted F1, class-specific precision and recall, calibration and consequential misclassification counts. Existing logs are reported as recorded evidence, not as tests freshly rerun for this chapter.

End-to-end live database behaviour, target-device compatibility, user acceptance, accessibility, throughput and recovery required separate evidence. Source code and isolated passing tests were not treated as substitutes for those evaluations.

5.4.2 Functional test cases and recorded results

Table 5.6 links representative test scenarios to implemented capabilities. The test identifiers organise this chapter and are not asserted to be the approved Chapter 4 requirement identifiers. Expected outcomes come from test assertions, while observed verdicts come from the saved v3 verification log. The saved verification_checks.json file in the v3 experiment directory contained the suite outputs and exit codes.

Table 5.6 Functional test cases and results

ID and scope	Scenario and expected result	Recorded result
TC01 Input validation	Submit missing, malformed, non-finite and out-of-range values; reject them.	Pass: SVM contract validation test.
TC02 Output mapping	Vary class order; map each probability to its class label.	Pass: class-order test.
TC03 Model consistency	Compare saved artifact and API predictions; retain legacy API fields.	Pass: artifact parity and legacy-contract test.
TC04 Availability	Make the model unavailable; do not invent an AI-service prediction.	Pass: unavailable-model contract test.
TC05 Dataset integrity	Regenerate saved data and inspect predictors; preserve reproducibility and exclude hidden predictors.	Pass: dataset integrity test.
TC06 Leakage control	Inspect split manifest and scaler fitting; fit preprocessing on training data.	Pass: partition and scaler test.
TC07 Decision policy	Apply review policy; preserve original returned probabilities.	Pass: saved-policy and improvement tests.
TC08 Backend validation	Supply incomplete scores; fail before inference or threshold fallback.	Pass: backend decision test.
TC09 Persistence contract	Return a model prediction; preserve database enums and decision basis.	Pass: backend decision test.

Table 5.7 Recorded suite totals

Suite	Tests	Result
test_svm.py	6	6 passed
test_kyc_experiment.py	2	2 passed
test_kyc_improvement.py	4	4 passed
backend/tests/kycDecision.test.js	2	2 passed
Total in saved v3 log	14	14 passed; zero recorded failures

These totals describe one retained verification log. They do not cover every test file in the repository or certify current uncommitted interface changes. No complete defect register and chronological retest record accompanied these results; consequently, no defect-resolution counts were inferred. Live registration, capture, administrator review and access enforcement remained outside the conclusions supported by this isolated log.

5.4.3 Model evaluation results

Both the saved v2 incumbent and selected v3 model were evaluated on the same 1,500 fresh synthetic cases. Using a common evaluation set avoided presenting the older v2 result from a different sample as a direct before-and-after comparison. The v3 experiment summary and improvement report contained the metrics presented in Tables 5.8–5.10.

Table 5.8 Model comparison on the same fresh evaluation set

Metric	Saved v2	Selected v3
Accuracy	75.733%	75.800%
Macro F1	0.651909	0.654082
Weighted F1	0.730391	0.731979
Rejected predicted Verified	44	43
Manual Review predicted Verified	186	184
Log loss	0.660123	0.659596
Multiclass Brier score	0.373659	0.373110

The accuracy difference was 0.067 percentage points, equivalent to one additional correct decision. The paired-bootstrap 95% interval for the accuracy difference extended from −0.133 to +0.333 percentage points. The macro F1 difference was 0.002173, with an interval from −0.001051 to +0.006855. Both intervals included zero, so the results did not establish a statistically conclusive improvement. They described uncertainty for the fitted models under the same simulator, not across real-world populations or all possible training runs.

Table 5.9 Selected v3 confusion matrix

Actual class	Predicted Verified	Predicted Review	Predicted Rejected
Verified	831	15	54
Manual Review	184	82	34
Rejected	43	33	224

Table 5.10 Selected v3 class-specific performance

Class	Precision	Recall	F1
Verified	0.785444	0.923333	0.848825
Manual Review	0.630769	0.273333	0.381395
Rejected	0.717949	0.746667	0.732026

The weakest class was Manual Review: only 82 of 300 cases were retained for review, while 184 were verified. In addition, 43 of 300 rejected cases were verified. Combining these errors gave 227 false verifications among 600 non-Verified cases, or 37.83%. Of all 1,058 predicted verifications, 227 were incorrect under the synthetic labels, or 21.46%. These derived rates have different denominators and should not be confused with the 24.20% overall classification error.

5.4.4 Error analysis and interpretation

The class imbalance and overlapping scenarios helped explain why accuracy alone overstated useful review performance. Verified cases represented 60% of the fresh sample and had high recall, whereas review cases frequently resembled verified cases in the observed feature space. The simulator deliberately included indistinguishable components with different latent labels. The stored identifiability analysis attributed a minimum eight-percentage-point population error contribution to those components, with further overlap potentially adding error. This was a property of the simulation, not an estimated limit for real identity verification.

Table 5.11 Permutation importance for the selected model

Feature	Macro F1 decrease	Repeat standard deviation
Liveness	0.201380	0.008477
Face match	0.110442	0.005995
OCR agreement	0.049802	0.007588
Blur	0.003831	0.003357
Brightness	−0.000359	0.003924
Contrast	−0.003439	0.003605

Liveness and face matching had the strongest measured influence on classification under permutation. OCR provided additional information, while brightness and contrast showed negligible or slightly negative average importance. Negative values meant that shuffling slightly improved the measured score in that experiment; they did not prove that image quality was useless. Quality also affected capture and extraction before the classifier, and correlated features could mask one another's contribution. The saved permutation-importance results recorded the mean changes and variation across repeats.

Log loss and Brier score improved slightly, but probability calibration remained conditional on the simulated class mixture. A score interpreted as confidence under a 60/20/20 synthetic distribution was not automatically a trustworthy probability for a different applicant population. Errors that grant access to a non-Verified case also carried a different operational consequence from errors that direct a genuine applicant to review. The evaluation therefore retained the two false-verification counts alongside aggregate metrics.

5.5 Version Control and Implementation Evidence

Project repository

The repository contained separate frontend, backend, AI-service and database directories, setup instructions, dependency files, SQL scripts, tests and saved experiment reports. Recent history included commits 1119c41, describing OCR changes, and 9d0fe2c, describing README work; the inspected HEAD was 870b303. Some current interface and passkey changes were uncommitted, so that commit alone did not identify every file described in the working implementation.

Legacy model artifacts were retained separately from the active v3 bundle, supporting comparison and rollback. Generic commit messages limited the explanatory value of parts of the history. Branch reviews, issue tracking, release tags, repository licensing and supervisor access were not established by the inspected evidence and were not claimed as completed practices.

5.6 Author Oversight and Use of AI Assisted Tools

OpenAI Codex assisted with preparation of this chapter by inspecting the project source, organising the supplied Chapter 5 guidelines, and drafting descriptions and tables from saved implementation and evaluation evidence. The draft distinguished recorded results from unverified deployment and user-evaluation claims. The student remains responsible for checking the chapter against the approved objectives, confirming the evidence and citations, and approving the final submission. This declaration does not assert that student or supervisor review has already occurred.

5.7 Evaluation Discussion and Limitations

5.7.1 Assessment of implemented capabilities

The implementation supported an integrated evidence-processing architecture: agents could be represented in the database, document and selfie components supplied inputs, separate services produced six scores, and decision outputs carried model or fallback provenance. The saved contract and backend tests supported correctness of selected interfaces and invalid-input handling. They did not establish that the complete live workflow met every approved requirement.

For automated classification, the available evidence supported a narrower finding: v3 achieved 75.800% accuracy and 0.654082 macro F1 on the declared fresh synthetic sample and passed its comparative release gate. The small improvement over v2 was inconclusive, while low review recall and 227 false verifications showed that the gate did not establish safe autonomous approval. Human-review facilities were implemented, but the experiment did not measure whether human reviewers corrected those errors.

For traceability and access control, linked score, decision, access and audit records provided an implemented basis for reconstructing verification actions. Role checks, token validation and retention logic added controls. Their presence did not establish resistance to every attack, complete privacy compliance or successful recovery under failures. A full-system acceptance verdict could not be derived from the recorded model and contract tests alone.

5.7.2 Limitations affecting the findings

Data validity was the main limitation. Synthetic adjudication labels, fixed class quotas and assumed score relationships constrained generalisation. The fresh sample came from the same simulator rather than independent real applicants. Historical benchmark measurements were reused, and changes to the current image pipeline could create mismatch between measured source distributions and deployed scores.

Evaluation coverage was also limited. No demographic fairness study, modern presentation-attack campaign, target-camera comparison, prospective pilot, participant-based usability study or measured end-to-end latency and throughput result was established. Minimum hardware, cloud capacity and long-duration reliability were therefore unresolved. The small calibration and selection partitions also left a risk of overfitting candidate policies.

Finally, classifier metrics excluded application quality gates, threshold fallback and administrator intervention. Unpinned dependencies and uncommitted changes limited exact reproduction of the complete system. The evidence supported a functioning implementation structure and a reproducible synthetic classifier experiment, while the extent of real-world effectiveness remained unestablished. Final conclusions, recommendations and future work belong in Chapter 6.